



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

Projektdokumentation im Rahmen vom Fach Datenverarbeitung in der Technik

VitaHelm - Der Helm der die Erregung misst

SoSe 25

Fakultät für Mathematik und Informatik

Ostbayerische Technische Hochschule Regensburg

Vorgelegt von: **Vinzenz Baumann, Albero König, Ronny Kenfack
Nzeko Prince**

Betreuer: **Matthias Altmann, Prof. Dr. Alexander Metzner**

Abgabedatum: **15.07.2025**

Inhaltsverzeichnis

1	Einleitung	1
1.1	Projektidee und Motivation(Vinzenz Baumann)	1
1.2	Zielsetzung	1
2	Technischer Überblick	1
2.1	Systemarchitektur(Vinzenz Baumann)	1
2.2	Verwendete Technologien	3
2.2.1	Verwendete Hardware Technologien (Napoleon Nzeko)	3
2.2.2	Verwendete Protokolle(Vinzenz Baumann)	3
2.2.3	Verwendete Softwaretechnologien	4
3	Hardware	6
3.1	Helmaufbau(Vinzenz Baumann)	6
3.2	Sensoren	8
3.2.1	Anschlüsse der Sensoren(Albero König)	8
3.2.2	Oxymeter (MAX30102) (Albero König)	8
3.2.3	Mikrofon (KY-038) (Ronny Kenfack)	9
3.2.4	Beschleunigungssensor (ADXL345) (Ronny Kenfack)	10
3.2.5	Kamera (AI Thinker ESP32 Cam Module) mit CHRI OV2640 160 Grad Weitwinkel (Vinzenz Baumann)	11
3.2.6	LED-Streifen (WS2812B) (Napoleon Nzeko)	11
4	Software	12
4.1	ESP32-Programmierung	12
4.1.1	Sensorabfrage(Vinzenz Baumann)	12
4.1.2	Mikrofon (Ronny Kenfack)	13
4.1.3	ADXL345 (Ronny Kenfack)	14
4.1.4	Oxymeter (Albero König)	14
4.1.5	UDP-Kommunikation zum PC(Vinzenz Baumann)	15
4.1.6	Empfang des Erregungswerts (Vinzenz Baumann)	16
4.1.7	LED-Ansteuerung im Abhängigkeit des Erregungswerts (Napoleon Nzeko)	16
4.1.8	LED-Steuerung in Abhängigkeit von Atem und Herzfre- quenz(Vinzenz Baumann und Albero König)	17
4.2	AI Thinker ESP32 Cam Module (Vinzenz Baumann)	18
4.3	PC-Software(Vinzenz Baumann)	19
4.3.1	Kommunikationsstruktur	19
4.3.2	UDP-Empfang der Sensordaten	19
4.3.3	Kamerastream-Verarbeitung (MJPEG)	20
4.3.4	OpenCV-Analyse	20

4.3.5	Clamping	20
4.3.6	Mapping (Skalierung)	21
4.3.7	Berechnung der Beschleunigungsmagnitude	21
4.3.8	Erregungswert	22
4.3.9	Visualisierung und Logging	24
5	Herausforderungen und Lösungen	25
5.1	allgemeine Stromverbrauch und Akkulösung (Ronny Kenfack)	25
5.1.1	LED-Stromversorgung (Napoleon Nzeko)	26
5.2	Atemerkennung(Vinzenz Baumann und Albero König)	26
6	Fazit und Ausblick	28
6.1	Verbesserungspotential (Ronny Kenfack)	28
6.2	Fazit (Vinzenz Baumann)	29
6.3	Ausblick	29
7	Stundenlisten	31
7.1	Stundenliste Vinzenz Baumann	31
7.2	Stundenliste Albero König	32
7.3	Stundenliste Ronny Kenfack	33
7.4	Stundenliste Nzeko Prince	34
8	Anhang	36
8.1	Quellen	36

1 Einleitung

1.1 Projektidee und Motivation(Vinzenz Baumann)

Die Verschmelzung von Mensch und Maschine ist aktueller denn je. Die Grenzen der Wearable Technologies werden von Unternehmen intensiv ausgereizt. Gerade Apple hat mit ihrem **Spatial Computer** einen hochpreisigen Vorstoß gewagt. Doch warum bei solchen Geräten aufhören? Bereits 2016 kündigte BMW mit dem Erscheinen ihres Concept Cars **BMW VISION NEXT 100**¹ ein Fahrzeug an, das die Stimmung des Fahrers mithilfe technischer Unterstützung nach außen übersetzt.

Beide Projekte haben jedoch nicht nur ihre hohen Entwicklungskosten gemeinsam, sondern kämpfen auch mit demselben Problem: Wie kann die Stimmung eines Menschen erfasst werden, wenn weder sein Gesicht noch sein Körper sichtbar sind?

Genau diese Problematik versucht der VitaHelm auf kostengünstige Art und Weise zu lösen.

1.2 Zielsetzung

Der Vitahelm misst sensorisch körpereigene Vitalwerte, verarbeitet diese Daten und kommuniziert nach außen, wie sich der Träger fühlt.

2 Technischer Überblick

2.1 Systemarchitektur(Vinzenz Baumann)

Die Systemarchitektur ist -dem Zeitaufwand des Projektes und den Kenntnissen der jeweiligen Projektbeteiligten angepasst- bewusst einfach gehalten.

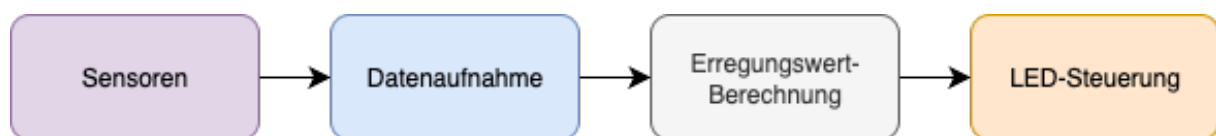


Abbildung 1: Datenflussdiagramm

Wie in Abbildung 2 dargestellt, lassen sich aus dem Flussdiagramm die grundlegenden Aufgaben der einzelnen Komponenten ableiten. Die Verteilung der Aufgaben erfolgt unter der Prämisse, dass Rechenleistung und Programmspeicher auf dem ESP32 eingespart werden sollen. Daher wird die vollständige Bildverarbeitung auf einen leistungsstarken Rechner ausgelagert. Der ESP32 übernimmt dabei hauptsächlich das Sammeln und Vorfiltern von Sensordaten sowie einfache Berechnungen und die Steuerung der LED. Zu seinen Aufgaben zählen auch kleinere Logikfunktionen und Animationen, die beispielsweise den Herzschlag und die Atmung visuell darstellen. Das AI Thinker Cam Modul

¹<https://www.bmw.at/de/topics/fascination-bmw/bmw-next-100/bmw-next-100.html>

stellt ausschließlich einen HTTP-Stream für den leistungsstarken Rechner bereit.

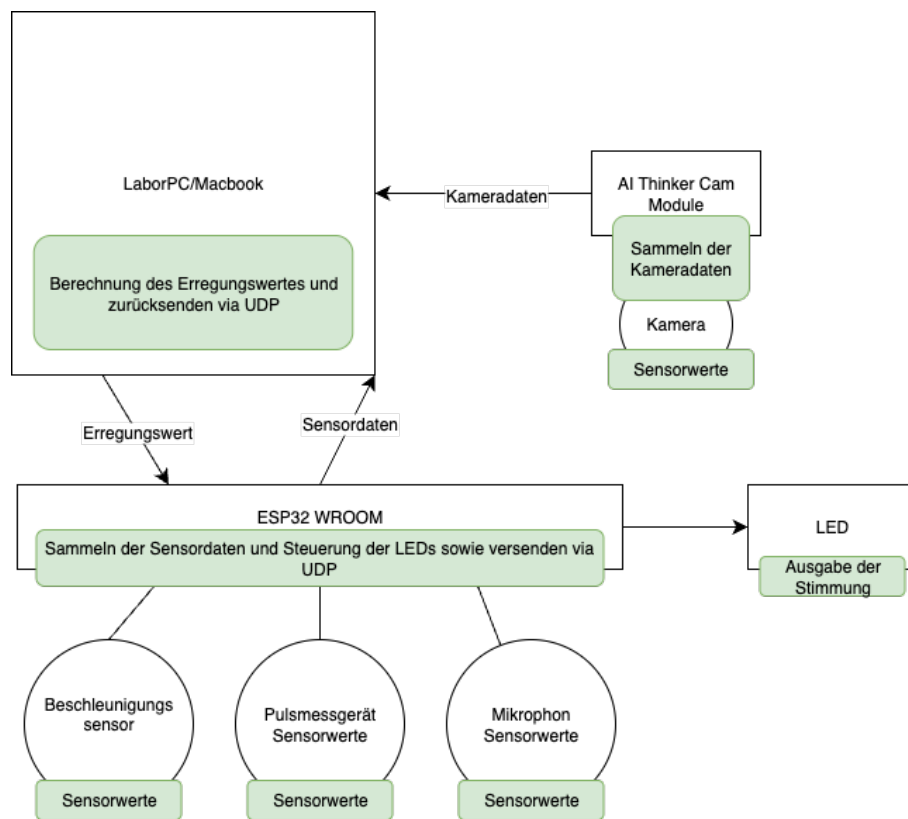


Abbildung 2: Aufgabenverteilung auf die einzelnen Komponenten

2.2 Verwendete Technologien

2.2.1 Verwendete Hardware Technologien (Napoleon Nzeko)

In diesem Projekt wurden folgende Hardwarekomponenten verwendet.

Tabelle 1: Verwendete Bauteile im Projekt

Bauteile	Bezeichnung / Typ	Funktion im Projekt
LED-Streifen(1)	WS2812B, 18 LEDs	Visualisierung von Erregung
LED-Streifen(2)	WS2812B, 3 LEDs	Gesichtsbeleuchtung
Microcontroller	ESP32 WROOM 32	Sammeln von Sensorwerten und Steuerung der LEDs
Microcontroller	AI Thinker ESP32 Cam Modul mit CHRI OV2640	Bereitstellen des Kamera-Streams
Pulsmessgerät / Oxymer	MAX30102	Pulsmessung in BPM
Mikrofon	KY-038	Erkennung von Ein- und Ausatmen
Beschleunigungssensor	ADXL345	Bewegungserkennung
Stromversorgung	5V-Netzteil	Versorgung aller Komponenten mit Spannung
Helm	Motorrad-Helm	Basis für das gesamte System
3D-Druckteile / verschiedene .stl files	Cases für Sensoren	Schutz und Halterung für Sensoren

2.2.2 Verwendete Protokolle(Vinzenz Baumann)

- Die Kommunikation zwischen dem **ESP32 WROOM** und den Sensoren erfolgt wie folgt:
 - **Pulssensor (MAX30105)** und **Beschleunigungssensor (ADXL345)** sind über das **I²C-Protokoll** angebunden.
 - Das **Mikrofon (KY-038)** wird über einen **analogen und digitalen GPIO-Pin** angeschlossen.
- Die vom ESP32 erfassten **Sensorwerte** (Puls, Atmung, Bewegung) werden per **UDP über WLAN** an einen **Labor-PC oder Mac** übertragen.
- Der **Erregungswert**, der auf Basis der Sensor- und Kameradaten auf dem PC berechnet wird, wird ebenfalls per **UDP** zurück an den ESP32 gesendet.
- Die **LED-Streifen** werden vom ESP32 mittels **serieller Datenleitung** angesteuert. Dabei werden Farbe und Intensität dynamisch angepasst, um den Erregungsstand visuell darzustellen.
- Die **AI Thinker ESP32-Cam** mit integrierter **OV2640-Kamera** sendet ein kontinuierliches **HTTP-Stream (MJPEG)** an den PC zur Echtzeit-Analyse der Pupillenposition.

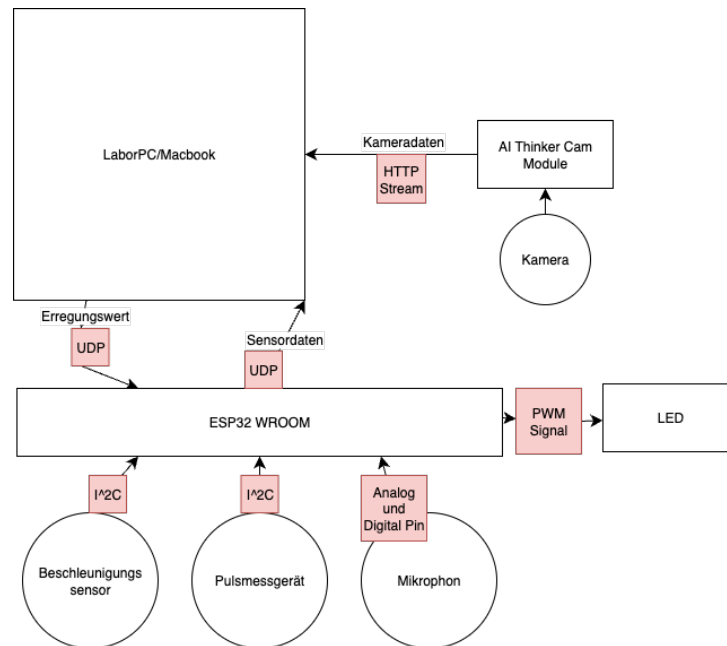


Abbildung 3: Verwendete Protokolle

2.2.3 Verwendete Softwaretechnologien

- **Programmiersprache:** Python 3 – Hauptsprache zur Umsetzung der Datenverarbeitung, Netzwerkkommunikation und Videoanalyse.
- **Datenkommunikation:** UDP (User Datagram Protocol) zur Übertragung von Sensordaten sowie zur Rückmeldung des berechneten Erregungswerts.
- **Bildverarbeitung:** OpenCV (Open Source Computer Vision Library) zur:
 - Verarbeitung und Darstellung eines MJPEG-Videostreams,
 - Anwendung eines Haar-Cascades zur Pupillenerkennung,
 - Rasterisierung des Bildes zur Bestimmung zulässiger Blickzellen.
- **Streaming:** Echtzeit-Empfang eines MJPEG-Kamerastreams über HTTP, realisiert mittels `urllib.request`.
- **Mathematische Analyse:**
 - Berechnung der Pupillenposition, -bewegung, -fläche und deren Veränderung,
 - Glättung der Werte durch exponentielle Gewichtung,
 - Normalisierung und Gewichtung mehrerer Parameter zur Ermittlung eines Gesamt-Erregungswerts,
 - Einfache geometrische Analysen (z.B. euklidische Norm).
- **Logging:** Speicherung aller relevanten Mess- und Analysewerte in CSV-Dateien mittels `csv`-Modul zur späteren Auswertung.

- **Verwendete Python-Bibliotheken:**

- `socket` – für die UDP-Kommunikation,
- `cv2` (OpenCV) – für Bildverarbeitung und Analyse,
- `urllib.request` – zum Einlesen des Kamerastreams,
- `math`, `time`, `os`, `csv`, `datetime`, `numpy` – für mathematische Berechnungen, Logging, Zeit- und Dateiverwaltung.

- **Verwendete LED-Bibliothek:** Ardafruit-Neopixel - zur Steuerung adressierbarer WS2812B-LEDs.

- **Versionsverwaltung:** Git und ein privates GitHub-Repository zur Verwaltung des Quellcodes und zur Sicherung der Entwicklungshistorie.

3 Hardware

3.1 Helmaufbau(Vinzenz Baumann)

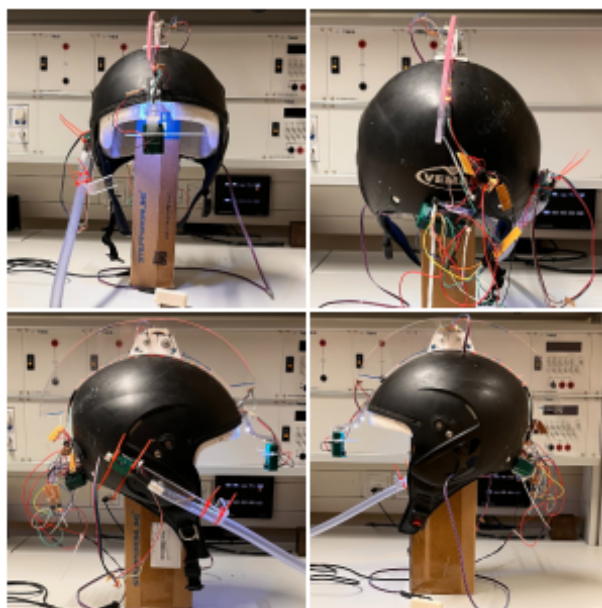


Abbildung 4: VitaHelm

Alle Sensoren sind am Helm angebracht, wobei die Umsetzung im Stil einer Prototypenentwicklung mit vorhandenen Materialien aus dem Labor erfolgt. Ursprünglich wird geplant, ein auf- und absetzbares Gesamtkonzept für den Helm zu konstruieren. Dazu wird mithilfe der iPhone-App Polycam ein 3D-Objekt erstellt und als STL-Datei exportiert, um die Rundung des Helmes für ein CAD-Programm zu berechnen. Die Konstruktion allein sprengt jedoch aufgrund begrenzter CAD-Kenntnisse den Rahmen. Manche Bauteile sind dynamisch anpassbar, sodass Positionsveränderungen bei Bedarf einfach vorgenommen werden können. Dies wird durch die Befestigung mit Kabelbindern realisiert.

Die LED-Finne besteht aus Plexiglas, das anhand einer selbst erstellten Schablone in eine annähernd passende Rundungsform gesägt wird. Die Form wird durch einen Durchleuchtungstest validiert, der für den Prototyp als ausreichend eingestuft wird.

Die Mikrofonhalterung wird aus einem zugesägten und mit einem Heizfön thermoplastisch gebogenen Plexiglasstreifen gefertigt und mit Schrauben am Helm befestigt. Daran wird das Mikrofon in einem 3D-Druck-Case mit Kabelbinder befestigt.

Für die Kamera wird zunächst eine Halterung aus leicht biegbarem Blech gebogen, um den Abstand zum Gesicht durch Tests zu ermitteln und erste Messdaten zu erhalten. Während dieser Phase entsteht die Idee, einen Teil der LED-Streifens zur Gesichtsbeleuchtung zu verwenden, um die Bildqualität zu verbessern und gleichmäßige Lichtverhältnisse zu schaffen. Anschließend wird in der gleichen Biegetechnik wie beim Mikrofon ein Plexiglasstreifen gefertigt, der die Kamera -ebenfalls mit Kabelbindern-an der korrekten Position fixiert.

Der Beschleunigungssensor wird so gerade wie möglich und zentral am oberen Helmrand direkt an der LED-Finne montiert, sodass die gemessenen Achsen mit der realen Orientierung übereinstimmen.

Für die Messung des Herzschlags am Zeigefinger wird ein 3D-Druckmodell aus dem Internet verwendet, das zusammen mit einer Feder und einer Schraube aus dem Labor montiert wird. Hierbei wurde ein Schrumpfschlauch verwendet um die Kabelbindung zu entlasten.

Der ESP32 wird am hinteren Teil des Helms befestigt. Alle Kabel werden zwischen Außenschale und Styropor des Helms geführt.

Die Lötarbeiten erfolgen nach Erstellung eines Schaltplans durch Vinzenz Baumann und werden anschließend durch Ronny Kenfack mittels Durchgangsprüfung kontrolliert.

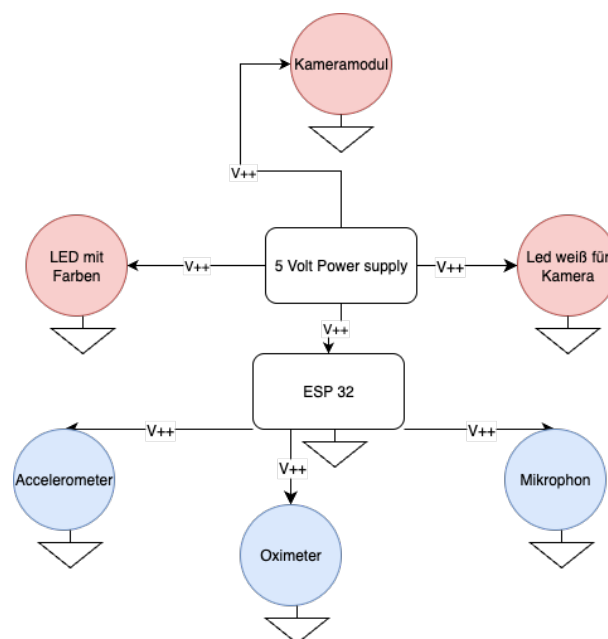


Abbildung 5: Power Supply und Schaltungsbild

Für alle verbauten Komponenten werden individuelle Gehäuse mittels 3D-Druck gefertigt. Da jedoch für viele Sensoren und Module keine exakt passenden Modelle verfügbar sind, erfolgt häufig eine nachträgliche Modifikation der Gehäuse.

Ein typisches Beispiel ist das Mikrofonmodul: Hier muss ein Zugang sowohl zum Potentiometer als auch zum eigentlichen Sensor offenbleiben, um spätere Justierungen oder Messmodifikationen zu ermöglichen.

3.2 Sensoren

Die Festlegung der einzelnen Parameter für die Berechnung einer Erregung zieht die Wahl der Sensoren nach sich.

3.2.1 Anschlüsse der Sensoren (Albero König)

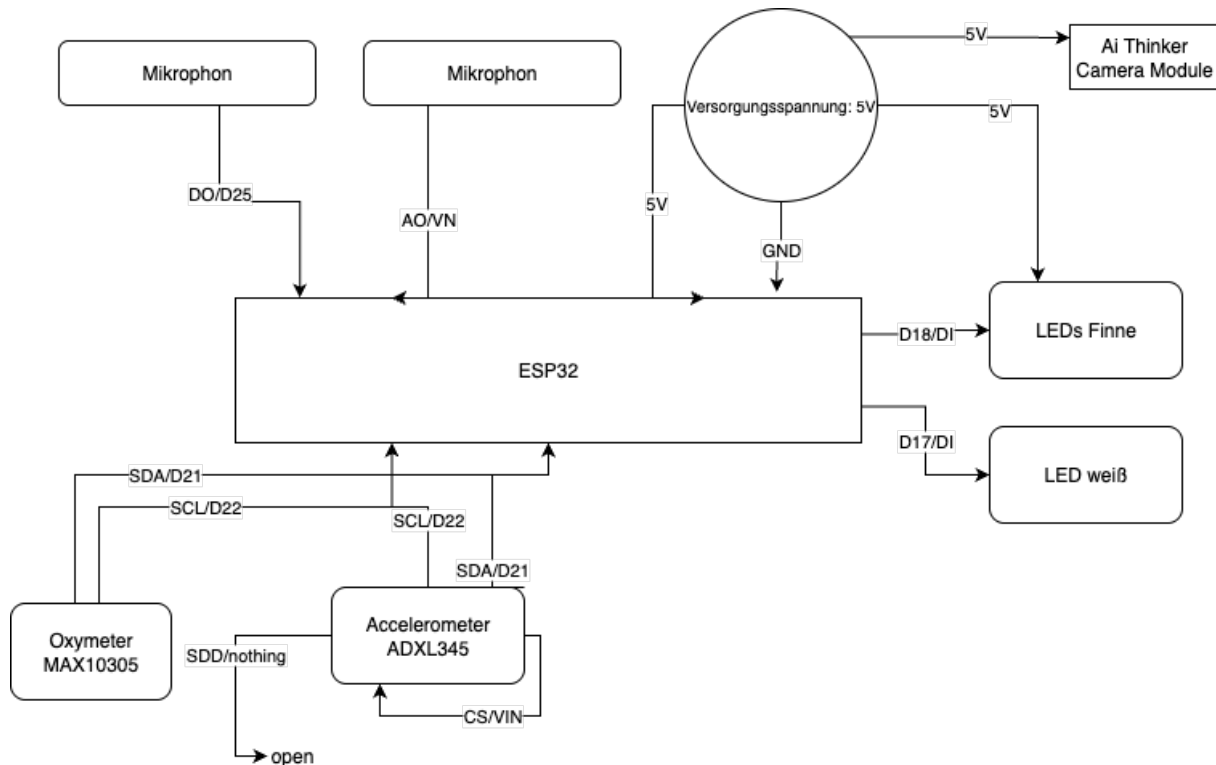


Abbildung 6: Anschlüsse der LEDs und Sensoren an esp

Alle Sensoren und LEDs sind grundsätzlich über ground und Versorgungsspannung angeschlossen. Dabei benötigen LEDs und Kamera Versorgungsspannung von 5V und andere Sensoren alle 3.3V. Oxymeter und Accelerometer benötigen Anschluss von SDA und SCL pins um I2C Verbindung herzustellen, diese liegen bei esp auf D21 und D22. Außerdem ist bei ADXL345 der VIN pin auf CS pin verbunden. Bei Mikrophon jeweils Digital und Analog pins auf D25 und VN an esp. Das Kameramodul liegt auf ihrem eigenen esp und überträgt Daten rein über udp.

3.2.2 Oxymeter (MAX30102) (Albero König)

Das Oxymeter wurde von uns zur Puls-Messung und Real Time Ausgabe des Puls an den LEDs verwendet. Wir haben uns für den MAX30102 entschieden da dieser sowohl real time Feststellung des Puls, als auch einfache Berechnung eines Durchschnittswertes erlaubt. Anfangs kam es zu Problemen bei Anschluss des Sensors, aufgrund von instabilem Anschluss der Kabel und später bei Integration ins Gesamtsystem durch Fehlerhafte Datenblätter und Problemen Bei Anschluss mit I2C. Testing erfolgte erstmalig nur mit dem

direkt an Kabeln angeschlossenem Sensor, welcher mit Gummibändern am Zeigefinger des Testers befestigt wurde. Anfangs war geplant die Pulsmessung am Ohr des Trägers des Helms durchzuführen, allerdings wurde im Laufe der Projektimplementierung beschlossen diese doch am Finger auszutragen, da die Anbringung am Helm dadurch sehr unpraktisch und instabil wäre. Die Messung der Daten mit dem MAX30102 erfolgt über einen Photosensor mit Hilfe von LEDs verschiedener Wellenlänge. Wir benötigen zur Pulsmessung nur die IR LED, diese strahlt Infrarotlicht aus, die Menge des reflektiertem Lichtes wird dann durch den Photosensor gemessen. Da Blut welches Sauerstoff trägt mehr Infrarotlicht absorbiert als Blut welches keinen Sauerstoff transportiert, kann anhand der an den Photosensor gelieferten Daten festgestellt werden wann ein Herzschlag geschieht.

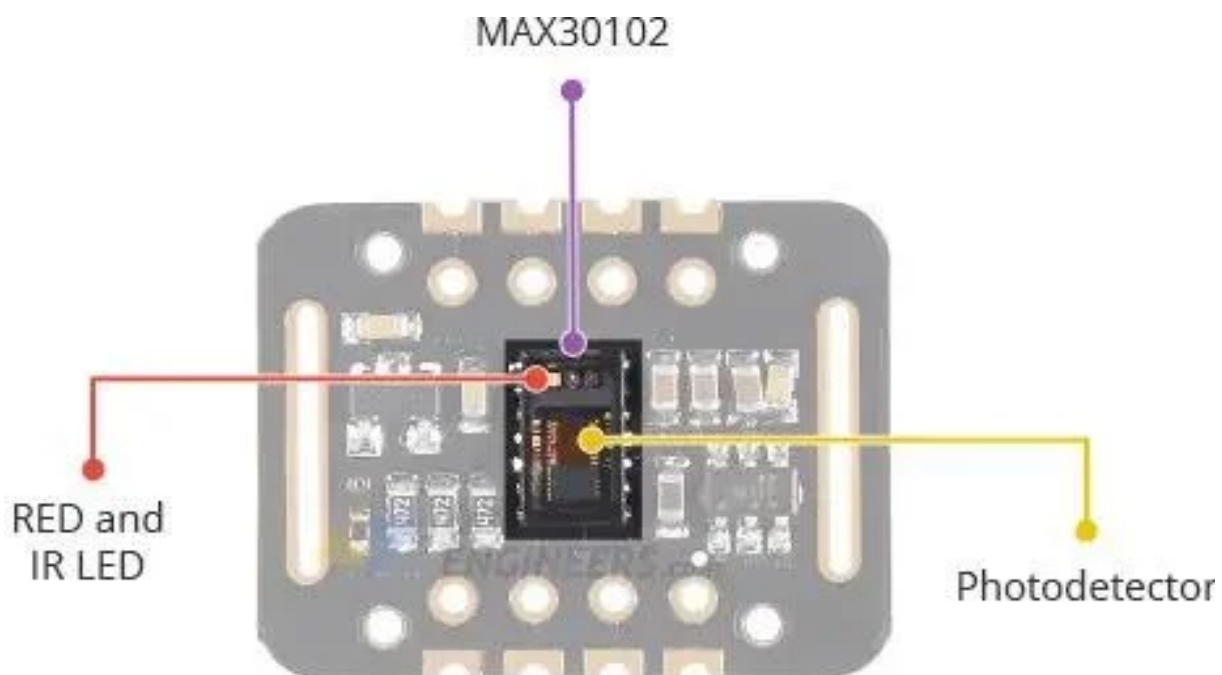


Abbildung 7: LEDs und Photosensor Oxymeter

2

3.2.3 Mikrofon (KY-038) (Ronny Kenfack)

Das KY-038-Modul dient zur Erfassung der Atemfrequenz durch digitale Flankendetektion. Rückblickend erweist sich die Wahl des KY-038 als suboptimal, da Atmungsgeräusche mit Pegeln von etwa 10 dB bis 20 dB weit unterhalb der detektierbaren Analogspanne von rund 48 bis 66 dB liegen. Daher haben wir die analoge Auslesung aufgegeben und fokussieren uns ausschließlich auf den digitalen Ausgang, um über die integrierte Komparatorschwelle jeden Einatmen- und Ausatmentakt als Flanke zu detektieren. Die Empfindlichkeit wurde per Potentiometer so weit angepasst, dass die Atemfrequenz ideal erfasst wird, ohne dass zu viele Umgebungsgeräusche fälschlich ausgelöst werden. Um die Sensoranfälligkeit für Störgeräusche zu minimieren, wurde das Mikrofon mit einem 3D-

²<https://www.instructables.com/Guide-to-Using-MAX30102-Heart-Rate-and-Oxygen-Sens/>

gedruckten Mundstück und Schrumpfschlauch adaptiert, wodurch eine bessere Isolierung und Filterung erreicht wird.

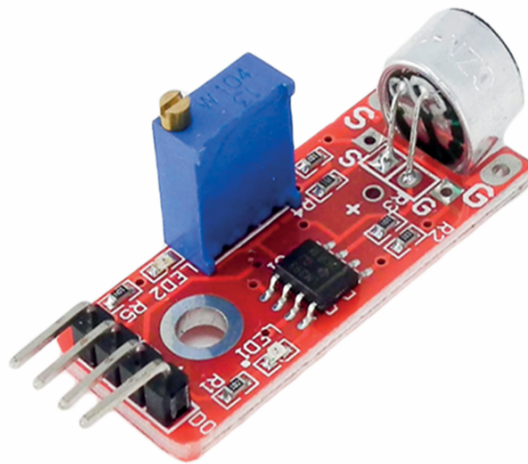


Abbildung 8: MiKrofon KY-038

3.2.4 Beschleunigungssensor (ADXL345) (Ronny Kenfack)

Das ADXL345-Modul misst Kopfbewegungen in drei Achsen mit einem maximalen Bereich von ± 16 g. Wir haben den Messbereich auf ± 8 g reduziert, um sowohl sehr schnelle Kopfstöße ohne Übersteuerung zu erfassen als auch feinste Neigungsänderungen abzubilden. Zuvor testeten wir den stromsparenden ADXL362, dessen niedrigere Abtastrate und höheres Rauschen jedoch eine saubere Auswertung ruckhafter Bewegungen verhinderten. Der ADXL345 sitzt exakt auf der Helmoberkante, wodurch er eine stabile Referenz behält und seine Achsen stets parallel zum Erdboden ausgerichtet sind. In der Funktion `getAccelerometerData()` wandeln wir die gemessenen m/s^2 -Werte mit dem Faktor 100 in 16-Bit-Ganzzahlen um. Anschließend sammelt `getFilteredAccelerometerData()` die letzten fünf Messungen in einem Ringpuffer und berechnet daraus einen gleitenden Mittelwert, der kurzzeitige Ausreißer zuverlässig glättet.

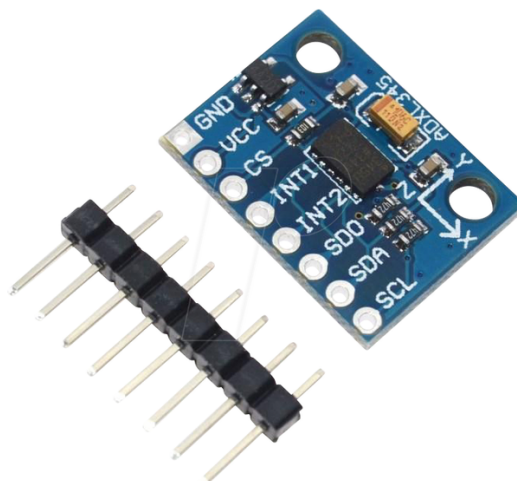


Abbildung 9: Beschleunigungssensor ADXL345

3.2.5 Kamera (AI Thinker ESP32 Cam Module) mit CHRI OV2640 160 Grad Weitwinkel (Vinzenn Baumann)

Die Wahl fällt auf das AI Thinker ESP32-CAM-Modul mit einer OV2640 160 Grad Weitwinkelkamera.³ Nach ersten Tests mit der Arduino IDE lässt sich das Board nicht über den mitgelieferten Controller flashen. Daher erfolgt der Wechsel zu PlatformIO in Visual Studio Code mit den passenden Bibliotheken und Frameworks. Das Flash-Problem besteht weiterhin, bis ein im Labor gefundener FTDI-Adapter zum Einsatz kommt, mit dem das Flashen problemlos funktioniert. Trotz korrekt aufgebautem HTTP-Stream lässt sich der Stream auf einem MacBook (M1, Chrome) nicht anzeigen, obwohl er auf dem Labor-PC funktioniert. Firewall oder Browserwechsel lösen das Problem nicht. Weiter fällt die Entscheidung auf die OV2640 Weitwinkelkamera mit 160° Sichtfeld⁴, um das gesamte Gesicht erfassen zu können. Dies ist besonders wichtig, da ursprünglich eine Mimikerkennung implementiert werden soll. Die Nähe der Kamera zum Gesicht erfordert daher eine solche Weitwinkeloptik, um alle relevanten Gesichtspartien vollständig im Bild zu erfassen. Das Kameramodul ist hierbei selbst WLAN fähig und liefert einen HTTP-Stream.

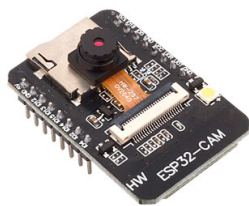


Abbildung 10: Ai Thinker ESP32 Cam Module links



Abbildung 11: OV2640 Kamera mit 160 Grad Weitwinkel

3.2.6 LED-Streifen (WS2812B) (Napoleon Nzeko)

Für die visuelle Darstellung der Vitaldaten fiel die Wahl auf adressierbare RGB-LEDs des Typs WS2812B, welche sich einzeln über eine serielle Datenleitung steuern lassen. Im Projekt wurden zwei LED-Streifen eingesetzt: ein Hauptstreifen mit 18 LEDs zur dynamischen Darstellung von Stimmung sowie ein zweiter, kürzerer Frontstreifen mit 3 LEDs zur konstanten Beleuchtung im Sichtfeld. Die Ansteuerung der LEDs erfolgt über zwei separate GPIO-Pins des ESP32, die Stromversorgung über ein externes 5V-Netzteil sorgt für ausreichend Leistung ohne Spannungseinbrüche.

³https://www.reichelt.de/de/de/shop/produkt/entwicklerboards_-_esp32-kamera_2mp_25_-266036

⁴https://m.media-amazon.com/images/I/61dHzX3midL._AC_SL1500_.jpg

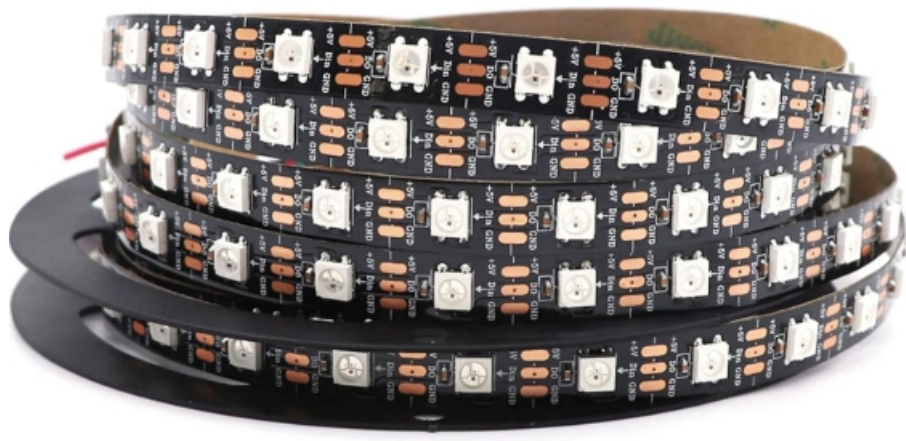


Abbildung 12: WS2812B-LEDs

4 Software

4.1 ESP32-Programmierung

Im Rahmen der Softwareentwicklung wurde die Projektstruktur bewusst gemäß dem klassischen Ansatz in Header-Dateien (.h) und Implementierungsdateien (.cpp) gegliedert. Diese Trennung fördert eine saubere Abstraktion der Schnittstellen und trägt wesentlich zur Übersichtlichkeit und Wartbarkeit des Gesamtsystems bei.

Die gewählte Struktur ermöglicht eine klare funktionale Trennung einzelner Module, wodurch die Komplexität reduziert und die Nachvollziehbarkeit einzelner Komponenten deutlich verbessert wird. Gleichzeitig erleichtert sie die zeitgleiche Arbeit mehrerer Entwicklerinnen und Entwickler am Projekt, da Schnittstellen definiert und Verantwortlichkeiten eindeutig zugeordnet sind.

4.1.1 Sensorabfrage(Vinzenz Baumann)

Im ESP32-Programm werden verschiedene Sensoren mit individuell abgestimmten Methoden abgefragt. Die Datenverarbeitung erfolgt innerhalb der Hauptschleife `loop()`, wobei ein Timerinterrupt mit 100 Hz (alle 10 ms) den Hauptdatenversand steuert. Die eigentliche Sensorabfrage erfolgt im Triggerfall (`sendD == true`):

- Die **Atmung** wird über ein digitales Mikrofon mit möglichst hoher Abtastrate gemessen. Die Funktion `updateBreathDetection()` verarbeitet diese Signale weiter und berechnet die Atemfrequenz in BPM mittels `getBreathRateBPM()`.
- Der **Beschleunigungssensor** (ADXL345) wird mit `getFilteredAccelerometerData()` ausgelesen. Die Rückgabewerte enthalten gefilterte Beschleunigungsdaten in X-, Y- und Z-Richtung.
- Die **Herzfrequenz** wird über ein Oximeter-Modul erfasst. Die Initialisierung und zyklische Abfrage erfolgen mit `oxymeterSetup()` und `oxymeterLoop()` aus einer

Bibliothek.

- Zusätzlich wird ein **digitaler Mikrofoneingang** zur Triggerdetektion per `digitalRead()` abgefragt.

Die Sensoren werden nicht mit identischer Pollingfrequenz abgefragt. Durch die Entkopplung der Messlogik (z. B. durch Zwischenspeicherung oder Mittelwertbildung) ist ein asynchrones, multitaskingfähiges Verhalten möglich. Dadurch können die individuellen Eigenschaften der Sensorik berücksichtigt und gleichzeitig eine zeitlich synchrone Datenausgabe erreicht werden.

4.1.2 Mikrofon (Ronny Kenfack)

Messung durch das Mikrofon

Der Mikrofon-Sensor liefert über den digitalen Pin ein binäres Signal, das bei Überschreiten einer definierten Lautstärke von LOW auf HIGH wechselt. Zusätzlich steht mit `readMicrophoneAnalog()` der analoge Rohwert zur Verfügung, um die Schaltschwelle dynamisch anzupassen und Umgebungsrauschen zu filtern.

Atemflankenerkennung

Die Funktion `updateBreathDetection()` liest kontinuierlich den aktuellen Pegel ein und erkennt steigende sowie fallende Flanken. Bei jeder steigenden Flanke wird das Flag `breathRisingEdge` gesetzt und die Zeit seit dem letzten Impuls erfasst. Entsprechend markiert eine fallende Flanke das Ende eines Atemzugs durch Setzen von `breathFallingEdge`.

Berechnung der Atemfrequenz

Aus den Zeitstempeln zweier aufeinanderfolgender Inspirationsanfänge errechnet `updateBreathDetection()` das Intervall zwischen den Atemzügen. Nur wenn dieses Intervall zwischen 0,9 Sekunden (66 BPM) und 5 Sekunden (12BPM) liegt, wird mittels der Formel

$$\text{BPM} = \frac{60000}{\text{Intervall in ms}}$$

die Atemfrequenz in Schlägen pro Minute bestimmt.

Signalweitergabe

Nach Erkennung eines Atemzyklus stehen die Flags `breathRisingEdge` und `breathFallingEdge` für nachfolgende Module bereit. Die Funktion `getBreathRateBPM()` gibt jederzeit den aktuellen BPM-Wert zurück und kann in der Hauptschleife oder in anderen Komponenten verwendet werden.

Analoge Signalauswertung

Über den Aufruf `readMicrophoneAnalog()` lässt sich das unbearbeitete Audiosignal auslesen. Diese Information ermöglicht eine manuelle oder automatische Kalibrierung der digitalen Schaltschwelle und erlaubt tiefergehende Analysen der Atemgeräusche.

4.1.3 ADXL345 (Ronny Kenfack)

Rohdatenerfassung

Die Funktion `getAccelerometerData()` greift über die Adafruit-Bibliothek auf den ADXL345 zu und liest ein aktuelles Beschleunigungs-Event aus. Die float-Werte in m/s^2 werden mit 100 multipliziert und in `int16_t` umgewandelt, um sie platzsparend weiterzuverarbeiten.

Filter-Buffer-Verwaltung

Im ringförmigen Puffer `filterBuffer` mit 5 Feldern wird bei jedem Aufruf von `getFilteredAccelerometerData()` ein neuer Messwert abgelegt. Der Index `filterIndex` läuft zyklisch und signalisiert durch `filterBufferFilled`, ob der Puffer bereits vollständig gefüllt ist.

Berechnung des gleitenden Mittelwerts

Nach dem Einfügen des neuesten Werts werden alle gültigen Einträge im Puffer aufsummiert. Die Summe der X-, Y- und Z-Komponenten wird durch die Anzahl der vorhandenen Messwerte (Puffergröße oder bisher gefüllte Elemente) geteilt. Dadurch entstehen geglättete Achsenwerte, die Störspitzen reduzieren.

Rückgabe gefilterter Beschleunigungswerte

Schließlich liefert `getFilteredAccelerometerData()` eine `AccelData`-Struktur zurück, die entweder den berechneten Mittelwert oder, falls noch keine Filterdaten vorliegen, den zuletzt gelesenen Rohwert enthält.

4.1.4 Oxymeter (Albero König)

Code examples wurden von SparkFun Electronics 2019 verwendet und auf eigene Bedürfnisse bearbeitet

Messung durch das Oxymeter:

- Das Oxymeter liefert kontinuierlich zwei Werte: IR und R.

Herzschlagerkennung:

- Mit der Funktion `checkForBeat()` wird anhand des IR-Wertes ein Herzschlag erkannt.

- Die Funktion prüft auf ein relatives Maximum und Minimum.
- Ein Herzschlag wird festgestellt, wenn die Differenz zwischen Maximum und Minimum bestimmte Kriterien erfüllt.

Signalweitergabe bei erkanntem Herzschlag:

- Wenn `checkForBeat()` einen Herzschlag erkennt, wird `heartBeatDetected` auf `true` gesetzt.
- Dieser Wert wird an `LED.cpp` übergeben.

Berechnung der Herzfrequenz (BPM):

- Die Zeit zwischen zwei erkannten Herzschlägen wird gemessen (`delta`).
- Aus `delta` wird die Herzfrequenz in Schlägen pro Minute (`beatsPerMinute`) berechnet.

Validierung und Speicherung der Herzfrequenz:

- Ein `beatsPerMinute`-Wert ist nur gültig, wenn er zwischen 20 und 255 liegt.
- Gültige Werte werden in das Array `rates[]` gespeichert.

Glättung des BPM-Wertes:

- Alle Werte im Array `rates[]` werden aufsummiert.
- Die Summe wird durch `RATE_SIZE` (Größe des Arrays) geteilt.
- Ergebnis ist ein geglätteter BPM-Wert

Anpassung der Messparameter aufgrund Fehlerhafter Messungen

- Oxymeter misst nicht mehr jeden Herzschlag aufgrund verschmutzten Photosensors
- Maximale Differenz für valide Herzschlagerkennung in `checkForBeat()` von 1000 auf 5000 erhöht

4.1.5 UDP-Kommunikation zum PC(Vinzenz Baumann)

Die Kommunikation mit dem PC erfolgt über das UDP-Protokoll. Der ESP32 agiert dabei als UDP-Client und sendet Daten an eine fest konfigurierte IP-Adresse und einen Port.

- **Heartbeat:** Alle 1000 ms wird ein kurzer Kontrollwert ("1") an den PC gesendet, um die Verbindung zu überprüfen.
- **Sensordatenpaket:** Alle 10 ms (100 Hz) wird ein strukturiertes Datenpaket per `udp.beginPacket()` an den PC übertragen. Die Datenstruktur ist textbasiert (CSV-ähnlich) und enthält:
 - Beschleunigungsdaten: X, Y, Z

- Mikrofontrigger (digital) und Mikrofonwert (analog)
- Atemfrequenz in BPM (**BreathRate**)
- Pulsfrequenz (BPM) und gleitender Mittelwert (**AvgBPM**)

Ein Beispiel für ein typisches UDP-Paket:

X:-50,Y:18,Z:1010,MicTrigger:0,MicAnalog:1234,BreathRate:12.5,BPM:78,AvgBPM:76

Diese Struktur erlaubt eine einfache und effiziente Auswertung auf dem PC, sowohl zur Visualisierung als auch zur weiteren Analyse und Klassifikation.

4.1.6 Empfang des Erregungswerts (Vinzenz Baumann)

Neben der unidirektionalen Übertragung von Sensordaten empfängt der ESP32 auch UDP-Nachrichten vom PC. Diese beinhalten einen berechneten **Erregungswert** im Bereich von 0 bis 600, der z. B. zur Steuerung von LEDs verwendet wird.

- Der Empfang erfolgt über `udp.parsePacket()` und `udp.read()`.
- Die empfangene Zeichenkette wird auf das Präfix `“Erregung:”` geprüft.
- Der numerische Wert wird anschließend extrahiert, validiert und in die globale Variable `erregungswert` gespeichert.

Diese Rückmeldung bildet die Grundlage für ein geschlossenes System mit Feedback-Loop: Die Sensorik misst biologische Parameter, der PC wertet diese aus und berechnet einen Erregungszustand, welcher wiederum vom ESP32 zur aktiven Steuerung verwendet wird.

4.1.7 LED-Ansteuerung im Abhängigkeit des Erregungswerts (Napoleon Nzeko)

Im Mittelpunkt der LED-Softwarelogik steht die Funktion `ledLoop()`, in der das LED-Verhalten basierend auf den Vitaldaten realisiert wird:

Farbzuweisung:

Der empfangene Erregungswert (`moodValue`) wird in der Funktion `updateMoodColor()` einer Farbzone zugeordnet (z.B. Blau = entspannt, Grün = erregt)

Berechnung des Farbwerts:

Der LED-Farbwert wird dann in Abhängigkeit der zugeordneten Farbzone und aus den verschiedenen Sensorsignalen (Puls, Atmung, Bewegung) berechnet. Beispiel für die rote Farbe:

```
r = moodR * globalBrightness;
```

`moodR`: Zielwert (abhängig vom Erregungsindex) `globalBrightness`: aus der Kombination von Basishelligkeit, Atmung und Herzschlag.

Farbübergänge:

Um Farbübergänge zu ermöglichen, wird die aktuelle Farbe an die neue Ziel-Farbe angepasst. Dies geschieht unter Berücksichtigung der Helligkeit.

Farblogik:

Die Farblogik orientiert sich an sechs definierten Erregungsbereichen:

Tabelle 2: Erregungslevel und zugehörige LED-Farben

Index	Kategorie	Wertebereich	LED-Farbe
1	entspannt	0–99	Blau
2	ruhig	100–199	Magenta
3	leicht erregt	200–299	Cyan
4	erregt	300–399	Grün
5	sehr erregt	400–499	Gelb
6	sehr sehr erregt	500–599	Rot

4.1.8 LED-Steuerung in Abhängigkeit von Atem und Herzfrequenz(Vinzenz Baumann und Albero König)

Der Code für die Umsetzung der visuellen Darstellung von Atmung und Herzschlag wird in spaßigen Pair programming Sessions von Vinzenz Baumann und Albero König erstellt. Die Steuerung des LED-Streifens basiert maßgeblich auf zwei physiologischen Parametern: der Herzfrequenz und dem Atemrhythmus. Beide Größen werden aus Sensordaten in Echtzeit extrahiert und in visuelle Rückmeldungen überführt.

Herzschlag: Der Herzschlag wird durch die Funktion `heartbeatDetected` erkannt, die über das `oxymeter.h`-Modul bereitgestellt wird. Sobald ein Herzschlag detektiert wird, wird der Helligkeitswert `heartBrightness` auf einen Maximalwert gesetzt (1.0) und anschließend über die Zeit hinweg linear auf null abgeblendet. Dies geschieht durch Berechnung eines Fade-Faktors in Abhängigkeit zur vergangenen Zeit seit dem letzten Herzschlag:

```
heartBrightness = 1.0 - (timeSinceBeat / fadeDuration);
```

Die resultierende Helligkeit `globalBrightness` ist eine Kombination aus diesem Wert und einer konstanten Basishelligkeit. Dadurch entstehen bei jedem Herzschlag kurze, weiche Lichtimpulse entlang des LED-Streifens, welche die Herzfrequenz visuell reflektieren.

Atmung: Die Atembewegung wird mithilfe eines digitalen Mikrofons erfasst. Bei Flankenwechseln des Mikrofonsignals (LOW zu HIGH) und ausreichender Zeitdifferenz (Entprellung über `debounceDelay`) wird zwischen Ein- und Ausatmung umgeschaltet:

```
if (lastMicState == LOW && currentMicState == HIGH) { ... }
```

Je nach erkanntem Atemzustand (`breathStateEinatmen`) wird ein laufflichtartiger Effekt über den LED-Streifen realisiert. Beim Einatmen wandert das Licht rückwärts (`currentLedIndex--`), beim Ausatmen vorwärts (`currentLedIndex++`). Dieser Effekt wird mit einer festen Rate aktualisiert und steht stellvertretend für die zyklische Natur des Atemvorgangs.

Durch die Kombination beider Eingangsgrößen entsteht ein Lichteffect, das sowohl die Tiefe als auch die Frequenz des Atems sowie den Puls visuell abbildet. Während die Herzschläge durch sanft pulsierende Helligkeitsänderungen dargestellt werden, sorgt der Atemrhythmus für eine kontinuierliche Bewegung entlang der LEDs. Diese Rückmeldung ermöglicht eine extrem intuitive Echtzeitwahrnehmung des physiologischen Erregungszustands.

4.2 AI Thinker ESP32 Cam Module (Vinzenz Baumann)

Der Kamera-Code implementiert eine einfache Streaming-Webanwendung für das ESP32-CAM Modul. Er basiert im Kern auf dem Beispielcode `CameraWebServer` aus der Arduino IDE, ist jedoch -wie im Folgenden beschrieben- reduziert, vereinfacht und gezielt angepasst.

- **Vereinfachter Webserver:** Anstelle des im Beispielcode genutzten `AsyncWebServer` (basierend auf `esp_http_server.h`) wurde der einfachere `WebServer` verwendet. Dies vereinfacht die Implementierung.
- **Minimierte HTML-Oberfläche:** Die komplexe Benutzeroberfläche mit Buttons, Auflösungswahl und Steuerfunktionen wurde entfernt. Stattdessen wird nur ein simples HTML-Dokument mit einem einzelnen `<img>` für den Livestream bereitgestellt.
- **Manuelle Streaming-Logik:** Die MJPEG-Übertragung wurde manuell mithilfe von `client.write()` und `server.setContent()` umgesetzt, anstatt über die Bibliothek `esp_http_server` zu arbeiten. Das erhöht die Kontrolle und Unabhängigkeit vom ESP-IDF um etwaige spätere Anpassung bezüglich timings durchführen zu können.
- **Statische WLAN-Konfiguration:** SSID und Passwort sind fest im Code hinterlegt. Funktionen wie automatisches WLAN-Setup, Access-Point-Fallback oder mDNS (`esp32cam.local`) wurden weggelassen.
- **Entfall optionaler Funktionen:** Nicht benötigte Features wie Bildaufnahme, LED-Flash-Steuerung, Auflösungswahl und OTA-Updates wurden entfernt, um den Code übersichtlich zu halten.

- **Reduzierte Fehlerbehandlung:** Die Fehlerbehandlung wurde auf eine einfache Ausgabe bei Kamera-Initialisierungsfehlern beschränkt.

Die Entscheidung fällt auf einen HTTP-Stream, da HTTP einfach zu implementieren ist und vom ESP32-Webserver direkt unterstützt wird. Zudem können nahezu alle Browser den MJPEG-Stream ohne zusätzliche Software anzeigen. Der Stream ist über eine URL im Browser einfach zugänglich und sollte problemlos durch Firewalls und Router funktionieren.

4.3 PC-Software(Vinzenz Baumann)

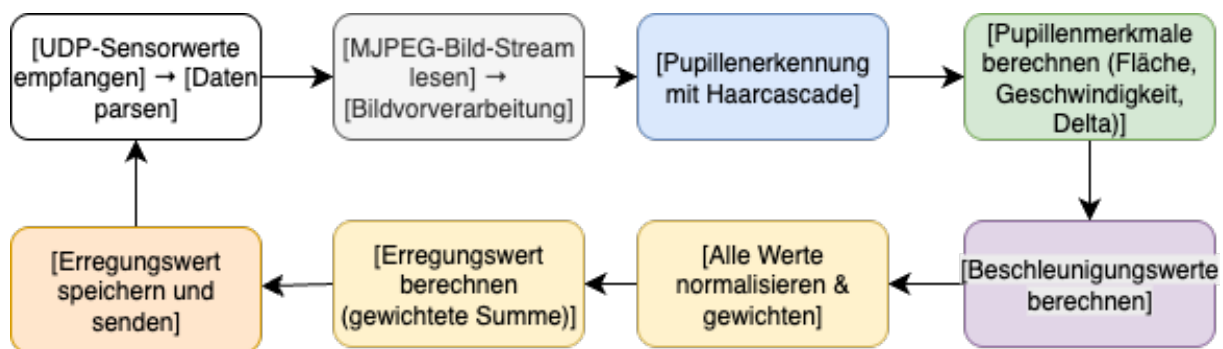


Abbildung 13: Ablauf der Erregungswertberechnung

Die Implementierung des obigen Ablaufes wird in der Programmiersprache Python umgesetzt. Nach anfänglicher Startschwierigkeiten mit der IDE Visual Studio Code und einigen Versionsproblemen auf dem MacBook M1 wurde ein virtuelles Environment eingerichtet, um den Python Compiler einstellen zu können. Hier wird das mit Python3 ausgeführt und hat ab diesem Zeitpunkt problemlos funktioniert.

4.3.1 Kommunikationsstruktur

Die Kommunikation zwischen ESP32 und PC erfolgt bidirektional über UDP. Der PC empfängt Sensordaten vom ESP32 und sendet berechnete Erregungswerte zurück an die Absenderadresse. Der Kamerastream wird über HTTP übertragen.

4.3.2 UDP-Empfang der Sensordaten

Der UDP-Empfang erfolgt über ein gebundenes Socket an IP 0.0.0.0 und Port 42190. Datenpakete werden kontinuierlich mit einem Timeout von 10 ms abgefragt, um Blockieren zu vermeiden. Empfangene Daten werden als String geparkt und die Sensorwerte wie X, Y, Z, MicTrigger, MicAnalog, BPM, AvgBPM und BreathRate extrahiert. Für Bewegungsdaten wird die Beschleunigungsmagnitude aus den dreidimensionalen Achsenwerten berechnet.

4.3.3 Kamerastream-Verarbeitung (MJPEG)

Der Kamerastream wird über HTTP von `http://172.16.12.215/stream` mittels `urllib.request.urlopen` geöffnet. Die Daten werden byteweise in einen Puffer eingelesen, und JPEG-Frames werden anhand der JPEG-Start- (`\xFF\xD8`) und -End-Markierungen (`\xFF\xD9`) extrahiert. Der Stream wird bei Verbindungsverlusten automatisch neu geöffnet.

4.3.4 OpenCV-Analyse

Jeder Frame wird gespiegelt und in Graustufen umgewandelt. Die Pupillenerkennung erfolgt mittels Haar-Cascade-Classifer aus der Bibliothek `openCV` (`haarcascade_eye.xml`). Gefundene Augen werden gefiltert, sodass nur die innerhalb erlaubter Rasterzellen (definiert durch `ALLOWED_CELLS` und `GRID_SPACING`) verbleiben. Für diese Augen werden Rechtecke gezeichnet und anhand von Mittelpunkten Positionen und Flächen berechnet, welche zur Bewegungsgeschwindigkeit und Flächendifferenz herangezogen werden.

4.3.5 Clamping

Sensoren in Echtzeitsystemen wie diesem Helm-Prototyp liefern kontinuierlich Datenströme. Diese Rohdaten sind jedoch häufig verrauscht, enthalten Ausreißer oder liegen außerhalb sinnvoller physiologischer Grenzen. Um eine zuverlässige Zustandsbewertung (*Erregungswert*) zu ermöglichen, ist eine Vorverarbeitung laut Heikenfeld et al. notwendig. (Heikenfeld u. a. 2018) Zwei zentrale Schritte dieser Vorverarbeitung sind das sogenannte **Clamping** und das **Mapping**.

Das Clamping dient der Begrenzung von Eingangswerten auf realistisch definierte Minima und Maxima. Es verhindert, dass fehlerhafte oder ungewöhnliche Sensorwerte unkontrolliert in die Erregungswert-Berechnung eingehen.

Die Implementierung erfolgt über eine einfache Funktion:

```
def clamp(value, min_val, max_val):  
    return max(min_val, min(max_val, value))
```

Diese Funktion erfüllt folgende Logik:

- Wenn `value < min_val`, wird `min_val` zurückgegeben.
- Wenn `value > max_val`, wird `max_val` zurückgegeben.
- Ansonsten bleibt `value` unverändert.

Beispielanwendung:

```
breath_rate = clamp(breath_rate, 5, 50)  
avg_bpm = clamp(avg_bpm, 40, 200)
```

Diese Grenzen wurden basierend auf physiologischen Normwerten und Erfahrungswerten gewählt (Tanaka, Monahan und Seals 2001 und StatPearls 2022):

- Die Atemfrequenz wird auf 5–50 Atemzüge pro Minute begrenzt.
- Die Herzfrequenz (BPM) wird auf 40–200 Schläge pro Minute begrenzt.

Ohne Clamping könnten einzelne Ausreißer durch Rauschen oder Sensorfehler die gesamte Erregungsbewertung verzerren, was zu Fehlinterpretationen oder Fehlreaktionen (z. B. unpassender LED-Ausgabe oder ungewolltes Springen des Erregungswertes) führen würde.

4.3.6 Mapping (Skalierung)

Das Mapping dient dazu, die einzelnen Sensorenwerte in ihre Scores anhand einer vorgefertigten Skala einzuordnen.

Die Mapping-Funktion hat folgende Form:

```
def map_range(value, from_min, from_max, to_min, to_max):
    return
    to_min +
    (float(value - from_min) / float(from_max - from_min))
    * (to_max - to_min)
```

Die Funktion berechnet eine lineare Abbildung von einem Ursprungsintervall $[a, b]$ auf ein Zielintervall $[c, d]$.

Beispiel:

```
breath_score = map_range(clamp(breath_rate, 5, 50), 5, 50, 0, 1)
```

4.3.7 Berechnung der Beschleunigungsmagnitude

Für die Bewertung der Beschleunigungsintensität ist zu berücksichtigen, dass Beschleunigungswerte sowohl positive als auch negative Vorzeichen annehmen können. Da die Bewegungsrichtung für die Anwendung jedoch nicht von Relevanz ist, wird die euklidische Norm verwendet, um die Gesamtintensität der Bewegung unabhängig von deren Richtung zu bestimmen.

Zunächst erfolgt eine Korrektur der Rohdaten des ADXL345-Sensors, da im Ruhezustand die Erdbeschleunigung (ca. $9,81 \text{ m/s}^2$) insbesondere auf der Z-Achse aufgeprägt ist. Durch Subtraktion dieses statischen Anteils wird sichergestellt, dass der Sensor im idealen Ruhezustand einen Wert nahe null liefert.

Gegeben sind zwei aufeinanderfolgende Beschleunigungsvektoren im dreidimensionalen Raum:

$$\vec{a}_{\text{neu}} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}, \quad \vec{a}_{\text{alt}} = \begin{pmatrix} x_{\text{alt}} \\ y_{\text{alt}} \\ z_{\text{alt}} \end{pmatrix}$$

Die Differenz der Beschleunigung in jeder Achse pro Zeitintervall $\Delta t = 0,01 \text{ s}$ (entspricht 100 Hz) ist:

$$\Delta x = \frac{x - x_{\text{alt}}}{100}, \quad \Delta y = \frac{y - y_{\text{alt}}}{100}, \quad \Delta z = \frac{z - z_{\text{alt}}}{100}$$

Diese Werte beschreiben die Geschwindigkeitsänderung. Die euklidische Norm der Änderungsrate ergibt die **Betragsänderung der Beschleunigung pro Zeitintervall** (Dombrowski 1999), also:

$$|\Delta \vec{a}| = \sqrt{\left(\frac{x - x_{\text{alt}}}{100}\right)^2 + \left(\frac{y - y_{\text{alt}}}{100}\right)^2 + \left(\frac{z - z_{\text{alt}}}{100}\right)^2}$$

Dies entspricht der Geschwindigkeit der Änderung des Beschleunigungsvektors, also einem Maß für abrupte Bewegungen oder Erschütterungen. Obwohl der Sensor Werte von 16 g messen könnte, werden nur Werte von $+3g$ in die Berechnung mit einbezogen, da dies für Kopfbewegungen aus eigener Kraft völlig ausreichend ist. Zusätzlich wurde der Messbereich noch im Sensor selbst auf $+8 \text{ G}$ gestellt, sodass man zwei Sachen damit bewirkt. Zum einen erreicht man eine höhere Auflösung um kleinere Änderungen mit aufzunehmen und zum Anderen erhält dies die Möglichkeit einer Erschütterungserkennung bei Unfällen zu implementieren. Diese liegen im Wert von 6 bis 8 g.

4.3.8 Erregungswert

Der Erregungswert E ergibt sich als gewichtete Summe mehrerer Eingangssignale, die kontinuierlich aus unterschiedlichen Sensorquellen erfasst werden. Die Gewichtung basiert auf der jeweiligen Relevanz der Messgrößen in Bezug auf den menschlichen Erregungszustand. So beeinflusst beispielsweise die Herzfrequenz den Gesamtwert deutlich stärker als die Geschwindigkeit der Pupillenbewegung, da sie ein wesentlich zuverlässigerer physiologischer Indikator für Stress oder Aktivierung ist.

Diese Gewichtungsstrategie orientiert sich an den Erkenntnissen aus Lee u. a. 2024. Es ist jedoch zu beachten, dass dieser Ansatz mögliche Korrelationen zwischen den Eingangssignalen nicht berücksichtigt. Im Rahmen dieses Projekts wird zudem die Gewichtung der Pupillendaten bewusst nach unten angepasst, da diese – trotz Glättung – starke Schwankungen aufweisen. Die reduzierte Gewichtung führt zu einer stabileren und intuitiveren Ausgabe des berechneten Erregungswerts.

Die allgemeine Formel zur Berechnung des Erregungswerts lautet:

$$E = w_p \cdot P + w_{p2} \cdot P_2 + w_{p3} \cdot P_3 + w_b \cdot B + w_a \cdot A + w_h \cdot H \quad (1)$$

- E Erregungswert (Ergebnis)
- P Pupillengeschwindigkeit
- $P1$ Pupillengröße
- $P2$ Pupillengrößenänderung
- B Beschleunigungswert
- A Atemfrequenz
- H Herzfrequenzwert (BPM)
- w_i Gewichtung des jeweiligen Anteils ($w_p, w_b, \dots$), wobei gilt:

$$w_p + w_{p1} + w_{p2} + w_b + w_a + w_h = 1 \quad (2)$$

Diese Gewichtungen wurden experimentell bestimmt und entsprechend dem intuitiven Wechsel der Farbausgabe an den LEDs iterativ angepasst. Die gewichtete Summe geht nicht direkt aus dem Code hervor. Hier ist schon der apriori berechnete Wert mit eingebunden, sodass die Gewichtung nicht direkt in der Formel ersichtlich ist, aber anhand der Werte ausgelesen werden kann.

```

speed_score = map_value(speed, -100, 100, 0, 30)
area_score = map_value(area, 0, 4500, 0, 30)
delta_score = map_value(delta, -2000, 2000, 0, 60)
accel_score = map_value(accel_magnitude, 0, 5.2, 0, 120)
breath_rate_score = map_value(breath_rate, 4, 50, 0, 120)
bpm_score = map_value(avg_bpm, 40, 200, 0, 240)
#berechnungsformel des gesamtwertes
gesamtwert = int(speed_score + delta_score + area_score + accel_score + bpm_score + breath_rate_score)

```

Abbildung 14: Summe des Erregungswertes mit der versteckten Gewichtungen

Dadurch wird gewährleistet, dass der Einfluss der einzelnen Messwerte eine intuitiveren Ausgabe der LEDs nach sich zieht.

Um eine zuverlässige und stabile Darstellung zu gewährleisten, werden die Rohdaten zunächst geglättet. Die Glättung erfolgt über gewichtete gleitende Mittelwerte (Moving Average) mit unterschiedlichen Glättungsfaktoren (α), um auf die Charakteristik der einzelnen Messgrößen einzugehen. Die verwendeten Formeln lauten:

$$\begin{aligned}
 \text{score_smooth}_t &= 0,2 \cdot \text{score}_t + 0,8 \cdot \text{score_smooth}_{t-1} \\
 \text{speed_smooth}_t &= 0,5 \cdot \text{speed}_t + 0,5 \cdot \text{speed_smooth}_{t-1} \\
 \text{delta_smooth}_t &= 0,5 \cdot \text{delta}_t + 0,5 \cdot \text{delta_smooth}_{t-1} \\
 \text{area_smooth}_t &= 0,3 \cdot \text{area}_t + 0,7 \cdot \text{area_smooth}_{t-1}
 \end{aligned}$$

Hierbei ist α der jeweilige Glättungsfaktor, der den Anteil des aktuellen Messwerts gegenüber dem vorherigen geglätteten Wert bestimmt.

Die Glättung dient der Reduktion von Messrauschen und unerwünschten Schwankungen, um eine aussagekräftigere Analyse der physiologischen Erregung zu ermöglichen.

Auch der Erregungswert wird selbst noch einmal vorm versenden über UDP geglättet.

4.3.9 Visualisierung und Logging

Die Visualisierung der erfassten und geglätteten Messgrößen erfolgt in Echtzeit mittels einer graphischen Benutzeroberfläche (GUI). Dabei werden die aktuellen Werte von Erregungswert, Geschwindigkeit, Flächenänderung und weiteren Parametern kontinuierlich aktualisiert und übersichtlich dargestellt.

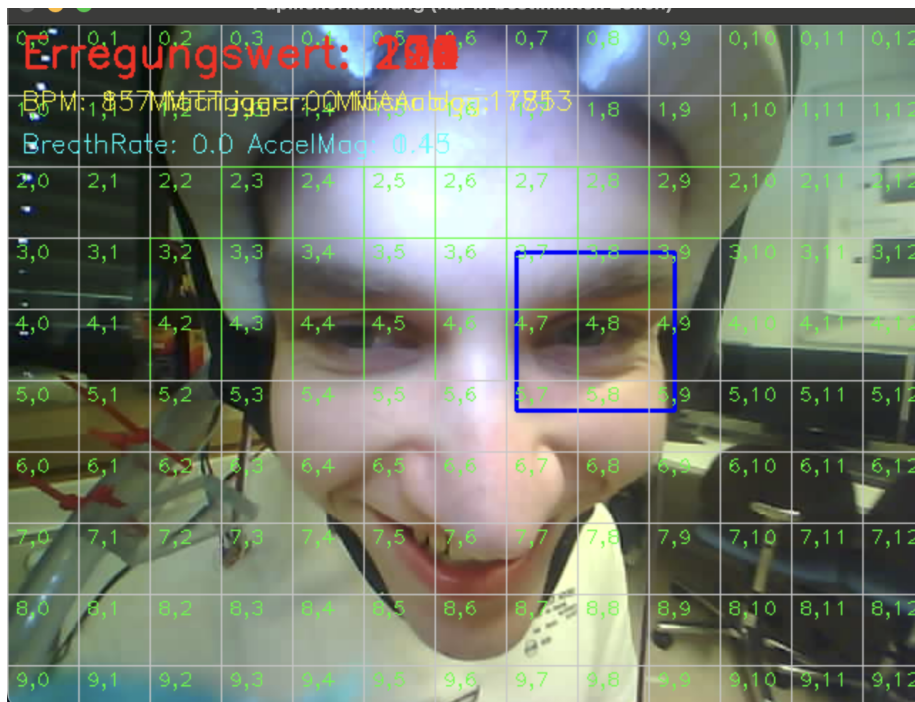


Abbildung 15: GUI

Parallel zur Live-Visualisierung erfolgt ein strukturiertes Logging aller relevanten Messgrößen im CSV-Format. Jede Zeile enthält einen Zeitstempel sowie alle geglätteten und verarbeiteten Daten, um eine nachträgliche Analyse und Auswertung zu ermöglichen.

Die Analyse dieser Logdateien diene wiederum als Grundlage für die Entwicklung der Gewichtungsfaktoren der einzelnen Teilkomponenten bei der Berechnung des Erregungswertes. Außerdem konnten durch Auswertung typischer und extremer Wertebereiche sinnvolle Clamp-Grenzen für Minimum und Maximum jedes Merkmals definiert werden. Dies ermöglichte eine normierte Einordnung und bessere Vergleichbarkeit zwischen unterschiedlichen Messsituationen.

Durch die Kombination von Live-Darstellung und systematischem Logging wird die unmittelbare Beobachtung des physiologischen Zustands gewährleistet.

5 Herausforderungen und Lösungen

5.1 allgemeine Stromverbrauch und Akkulösung (Ronny Kenfack)

Die Stromversorgung unseres Systems stellte ebenfalls eine Herausforderung dar, da die einzelnen Komponenten unterschiedliche Spannungen benötigen. Für die Testphase haben wir uns daher für ein externes Netzteil entschieden, das 5 V an das ESP32-WROOM-Modul, die ESP-CAM und den LED-Streifen (die Haupt Streife und die kleine vorne an der Kamera) liefert. Wohingegen der Pulssensor, das KY-038-Mikrofon und der Beschleunigungssensor, die 3,3 V benötigen, speisen wir direkt über die Spannungsregler des ESP32-WROOM.

Ursprünglich war geplant, den Helm mit einer internen Batterie zu betreiben – hierfür hätte man jedoch ein zusätzliches Montage- und Schutzgehäuse für den Akku benötigt. Alternativ haben wir untersucht, alle Module und Sensoren ausschließlich über das externe Netzteil bzw. die Batterie zu versorgen und das ESP32-WROOM nicht als Spannungsquelle zu nutzen. Obwohl ein Level-Shifter in der Testphase funktioniert, wurde diese Lösung im finalen Helmbau wegen der hohen Verkabelungskomplexität verworfen.

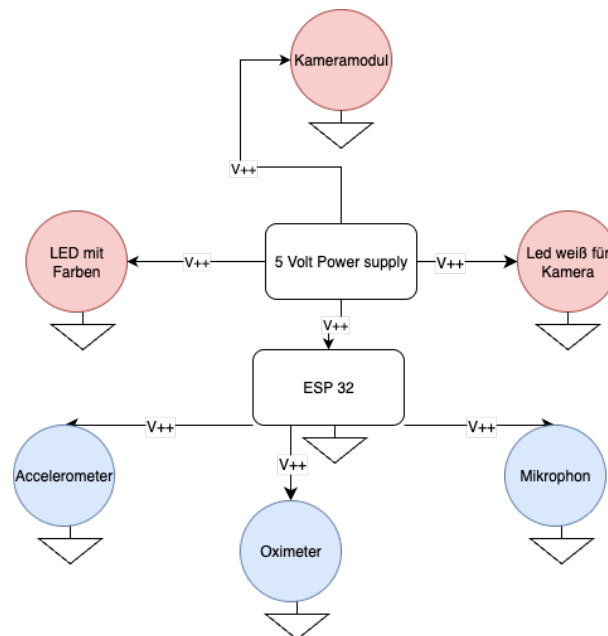


Abbildung 16: Power Supply und Schaltungsdiagramm

5.1.1 LED-Stromversorgung (Napoleon Nzeko)

Erste Tests zeigten, dass die 3,3V-Logikpegel des ESP32 nicht ausreichen, um den LEDs-Streifen zuverlässig anzusteuern. Das Problem äußerte sich durch Flackern und fehlende Signale.

Zur Lösung wurde der externe 5V-Netzteil eingesetzt, um die Stromversorgung der LEDs getrennt vom ESP32 zu erfolgen. Die GND-Leitung des Netzteils wurde dabei mit dem GND des ESP32 verbunden, um einen gemeinsamen Referenzpegel sicherzustellen. Dies hat ermöglicht, Spannungseinbrüche zu vermeiden, insbesondere bei hoher Helligkeit und gleichzeitiger Animation.

5.2 Atemerkennung (Vinzenn Baumann und Albero König)

Die Erkennung von Ein- und Ausatmung stellt aufgrund der gewählten Mikrofonlösung eine erhebliche Herausforderung dar. Während das Einatmen in der Regel geräuscharm aus einem größeren Volumen erfolgt, erzeugt das Ausatmen einen gerichteten, akustisch deutlicheren Luftstrom. Trotz gleicher Luftmengen unterscheiden sich die Signale beider Phasen signifikant.

Zur Differenzierung wird eine Schwellwtererkennung mit Toggle-Logik implementiert. Ein detektiertes Atemereignis schaltet zwischen Ein- und Ausatmung um, wobei die LED-Steuerung die Richtung der Animation entsprechend ändert. Auch wenn dies nicht in jedem Fall eine konsistente Animation garantiert, entsteht ein visuell ansprechendes und nachvollziehbares Feedback.

Da der Luftstrom nicht gleichmäßig verläuft, treten sprunghafte Änderungen in den Mikrofonwerten auf. Zur Stabilisierung wird ein dynamischer *Debounce*-Mechanismus eingeführt, der sich an die aktuell gemessene Atemfrequenz anpasst. Dies verhindert die doppelte Erkennung einzelner Atemphasen und verbessert die Signalverarbeitung auch bei schneller Atmung.

Iterativer Aufbau zur Optimierung Die initiale Lösung liefert keine zufriedenstellenden Ergebnisse. Daher wird ein alternativer Aufbau mit folgenden Zielsetzungen entwickelt:

- Gleichmäßiger Signalpegel bei Ein- und Ausatmung
- Stabiler Luftstrom zur sicheren Detektion
- Reduktion von Umgebungsgeräuschen

Eine veränderte Mikrofonposition im Nasen- oder Mundbereich führt nicht zur gewünschten Verbesserung. Inspiriert von der Funktionsweise einer Mundharmonika – bei der Ein- und Ausatmung sogar unterschiedliche Töne erzeugt – wird ein akustisch unterstützter Atemadapter konzipiert.

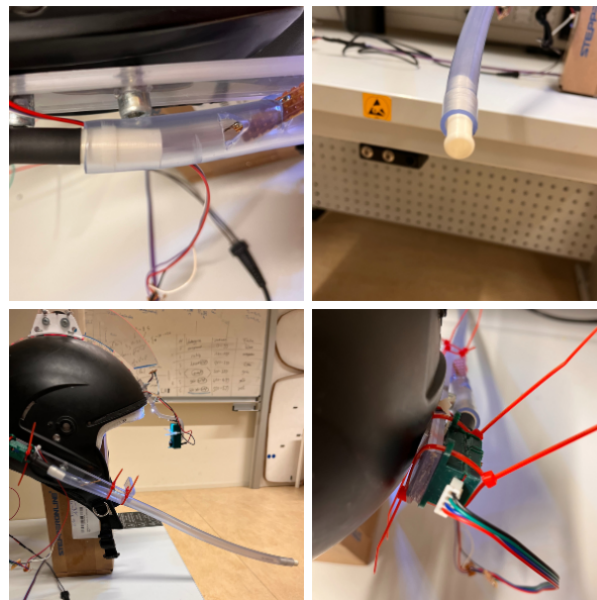


Abbildung 17: Mikrofon Aufbau

Der Aufbau besteht aus einem 3D-gedruckten Mundstück (Shisha-Mundstück), in dem das Mikrofon über einen Schrumpfschlauch fixiert ist. Zwei seitliche Öffnungen ermöglichen

eine nahezu druckfreie Atmung. Eine teilweise abdeckende Steckbrettplatine in einem der Löcher erzeugt ein leichtes Rauschen, das zuverlässiger vom Mikrofon erkannt wird. Durch das gezielte Führen des Luftstroms direkt zum Mikrofon wird die Signalqualität deutlich verbessert.

Zusätzlich ermöglicht der mechanische Aufbau eine gezielte Reduktion des Mikrofonpegels. Durch die abgeschirmte Umgebung im Inneren des Schlauchs wird der Einfluss von Umgebungsgeräuschen stark reduziert. Dies erhöht die Selektivität für relevante Atemsignale und ermöglicht eine robustere Signalerkennung. Trotzdem hätte man noch eine Art Popschutz um die Auslasslöcher einbauen können. Diesen hätten den Luftstrom nicht behindert, aber noch besser von äußeren Einflüssen abgeschirmt.

6 Fazit und Ausblick

6.1 Verbesserungspotential (Ronny Kenfack)

Erste Praxistests haben gezeigt, dass das System zwar zuverlässig misst, aber in puncto Bedienkomfort und Modularität noch Luft nach oben besteht. Mit gezielten Erweiterungen ließe sich der Helm noch intuitiver gestalten und seine Einsatzmöglichkeiten deutlich ausweiten:

Erweiterte Erregungsdetektion: Zusätzliche Gesichtsmimik-Analyse per Kamera für aussagekräftigere Stress- und Erregungsindikatoren.

Optimierte Atemfrequenzmessung: Einsatz eines dehnbaren Brustgurts (Respiratory Inductance Plethysmography) oder eines MEMS-Mikrofons mit differenzieller Kapsel zur Reduzierung von Umgebungsrauschen und stabileren Signalen.

Robustes Gehäuse und Kabelmanagement: Maßgefertigtes 3D-Druck-Gehäuse, integrierter Kabelkanal, wechselbarer Akku sowie Hauptschalter und Status-LEDs.

Nahtlose Pulsoximeter-Integration: Anstatt ein Fingerclip, Kann ein Ohrclip- oder Stirnband-Sensoren aufgebaut werden. Da wir alles an dem Helm integrieren wollen.

Intuitive Software und Benutzeroberfläche: Automatische Kalibrierungsroutinen beim Systemstart, begleitende Smartphone-App mit Live-Dashboard, Alarmfunktionen.

Begrüßungs- und Statusmeldung: Akustisches Feedback oder Sprachausgabe beim Aufsetzen („Willkommen, VitaHelm aktiv“) zur unmittelbaren Rückmeldung über den Systemstart.

Sprachsteuerung und -ansagen: Einfache Sprachbefehle („Start“, „Pause“, „Status“).

Sitz- und Komfortsensorik: Druck- oder Näherungssensoren prüfen die korrekte Helm-Position und geben Feedback bei Fehlausrichtung.

6.2 Fazit (Vinzenz Baumann)

Das Projekt VitaHelm war eine äußerst bereichernde Lernerfahrung, die nicht nur technisches Wissen, sondern auch wichtige Kompetenzen im Bereich Teamarbeit, Projektorganisation und Problemlösung vermittelt hat. Besonders hervorzuheben ist das gewonnene tiefere Verständnis für die Herausforderungen und Besonderheiten von Echtzeitsystemen, das durch die praktische Umsetzung mit verschiedenen Sensoren, der Kameraeinbindung und der Echtzeitkommunikation per WLAN deutlich geschärft wurde.

Die enge Zusammenarbeit im Team hat gezeigt, wie wichtig klare Kommunikation, gemeinsame Zielsetzung und strukturierte Aufgabenverteilung für den Projekterfolg sind. Dabei konnten wir nicht nur unsere technischen Fähigkeiten ausbauen, sondern auch lernen, wie man mit unvorhergesehenen Problemen konstruktiv umgeht und Lösungen im Team erarbeitet.

Insgesamt war das Projekt nicht nur lehrreich, sondern hat auch sehr viel Spaß gemacht. Die Kombination aus kreativer Hardwareentwicklung, Echtzeitdatenverarbeitung und interdisziplinärer Zusammenarbeit machte VitaHelm zu einem spannenden und motivierenden Projekt, das uns in vielerlei Hinsicht weitergebracht hat.

6.3 Ausblick

Der *VitaHelm* bietet durch seine modulare Sensorik, die Echtzeitdatenverarbeitung und die smarte Kommunikation mit externen Systemen vielfältige Einsatzmöglichkeiten über das ursprüngliche Projektziel hinaus. Besonders im Bereich der **Sicherheits- und Arbeitsschutzsysteme** könnte der Helm z. B. auf Baustellen, in gefährlichen Industrieumgebungen oder beim Katastrophenschutz eingesetzt werden, um Vitalparameter der Träger*innen zu überwachen und bei kritischen Werten sofortige Warnungen oder automatische Notfallmaßnahmen auszulösen.

Auch im **Sport- und Trainingsbereich** bietet der *VitaHelm* Potenzial – etwa zur Erfassung physiologischer Belastung, zur Optimierung von Trainingsprozessen oder zur Unfallprävention im Extremsport. In der **medizinischen Nachsorge** oder bei der **Begleitung von Risikopatient*innen im Alltag** könnte der Helm helfen, Veränderungen des Gesundheitszustands frühzeitig zu erkennen.

Nicht zuletzt ist auch ein Einsatz in **Forschung und Human Factors Engineering** (Wickens u. a. 2019) denkbar, z. B. zur Untersuchung von Stress, Aufmerksamkeit und Ermüdung in Echtzeit – sei es im Verkehr, in der Luftfahrt oder in der Mensch-Maschine-Interaktion.

Durch die flexible Erweiterbarkeit des Systems sind zukünftige Anpassungen und Weiterentwicklungen leicht möglich, was dem *VitaHelm* ein breites Spektrum an Anwendungsmöglichkeiten eröffnet.

7 Stundenlisten

7.1 Stundenliste Vinzenz Baumann

Stundenliste

Name: Vinzenz Baumann
Gruppe: 9, VitaHelm

Pos.	Bezeichnung	Beschreibung	Stunden in h
1	Integration in den Helm	Integration der Sensoren und Bauteile in den Helm	12
2	3D Druck	3D Druck für die Sensorencases und kleiner Assets	12
3	Bauteile	Bestellen, Suche und Recherche der Bauteile	3
4	Einscannen des Helmes	Scannen, exportieren und Laden eines CAD Programmes und	3
5	Erstellung der Abgaben	Projektbericht, Poster, Dokumentation	10
6	Fehlerbehebung	Fehlerbehebungen der Implementierung auf dem ESP32	18
7	Hilfe bei Teamkollegen	Hilfe bei Problemen in einzelnen Stadien im Projekt	8
8	Kamera	Programmcode auf Modul, Optimierung, Recherche	12
9	Kommunikation	Kommunikation mit Experten (Professoren)	1
10	LED Finne	Aussägen und Bearbeiten der Finne	2
11	Lötarbeiten	Lötarbeiten der Verkabelung	12
12	Mikrofon Aufbau	Problemlösung des Ein- und Ausatmens	6
13	Pair Programming	Zusammen mit Albero König Herzschatlogik , Atmunglogik	18
14	Programmieren Python	Programmcode für berechnung des Erregungswertes	20
15	Projektfindung	Idee und Projektfindung	3
16	Projektstruktur	Erstellen und pflege des git Repositories, Erstellen des Esp32 Projek	4
17	Recherche	Recherche für Codeexamples und Sensorverständnis	6
18	Steckbrettarbeit	Aufbau des Steckbrettes	6
19	Teamlead	Aufgabenverteilung und Koordination	10
20	Testing	Testen des helms in verschiedenen Stadien des Projektes	12
21	Vorstellungen	Präsentation und Ergebnisvorführung mit Vorbereitung	8
22			
23			
24			
25			
26			
27			
28			
29			
30			
31			
32			
33			
34			
35			
36			
37			
38			
39			
40			
41			
42			
43			
Gesamt:			186

Abbildung 18: Stundenliste Vinzenz Baumann

7.2 Stundenliste Albero König

Stundenliste

Name: Albero König
Gruppe: 9 VitaHelm

Pos.	Bezeichnung	Beschreibung	Stunden in h
1	Projektfindung	Auswahl des Projektthemas	3
2	Plakatbesprechung	Besprechung Plakatlayout gemeinsam/eigene Anteile	2
3	Bauteilliste	Erstellung und Besprechung der Bauteilliste	2
4	Gyrosensor	Nachforschung und Versuch erstes Ansprechen Gyro	5
5	Auswahl Gyro neu	Nachforschung und Auswahl eines neuen Gyros	4
6	Oxymeter	Nachforschung Funktionsweise Oxymeter	5
7	Code Umgebung	Einrichtung der Code Umgebung	2
8	Oxymeter integration	Erster Versuch Oxymeter Daten zu erhalten	5
9	Oxymeter fixing	Inkonsistente Daten von Oxymeter erhalten -> debug	6
10	Systemmigration	Umstieg von Arduino IDE auf VSCode	1
11	esp Integration	Integration von mehreren Sensoren auf einen esp32	6
12	Datenschnittstellen	Absprache Datenübergabe zwischen Oxymeter und PC	3
13	Codeanpassungen	Abänderungen Beispielcode auf eigene Bedürfnisse	5
14	Aufsetzen Projekt	Aufsetzen des Gesamtprojektes auf eigenem Laptop	3
15	Testing Oxy udp	Testen der Übergabe der Werte von Oxymeter auf PC	4
16	Oxymeter debugging	Problemlösung nicht funktionelles Oxy	5
17	Helmintegration	Anbau des esp und der Sensore and den Helm	10
18	Testing am Helm	Testen der Sensore nach Integration am Helm	7
19	Pair Programming	Ein- und Ausatmen Logik mit Vinzenz Baumann	10
20	Pair Programming	Herzschlag Ausgabe am Helm	8
21	Vorbereitung Vortrag	Erstellung Vortrag vor allen	5
22	Mikrophon Anbau	Problemlösung Anbau des Mikrofons	6
23	Erregungswerttesting	Testen und tweaken des Erregungswertes	8
24	Ergebnisvorstellung	Vorbereitung und Vorstellung Ergebnisse	7
25	Bearbeitung Gyro	Bearbeitung Sensorauswertung Accelerometer	8
26	smoothing werte	smoothing der Sensorwerte	3
27	Dialing Sensoren	Anpassung der Sensoren	5
28	Dokumentation	Erstellung der Dokumentation	10
29	recherchearbeit	Sammeln von Datenblättern	2
30			
31			
32			
33			
34			
35			
36			
37			
38			
39			
40			
41			
42			
43			
Gesamt:			150

Abbildung 19: Stundenliste Albero König

7.3 Stundenliste Ronny Kenfack

Stundenliste

Name: Ronny Roy Kenfack Zangue
 Gruppe: 9, VitaHelm

Pos.	Bezeichnung	Beschreibung	Stunden in h
1	Projektfindung	ProjektIdeen sammeln	4
2	Bauteilbeschaffung	Suche und Zusammenstellung aller benötigten Komponenten	3
3	Atemfrequenzmessung	Test des Mikrofons und Berechnung der Atemfrequenz über	17
4	Sensorvergleich	Evaluierung unterschiedlicher ADXL-, Pulsoximeter- und Mikr	12
5	Analogauslesen	Erfassung der Atemfrequenz über den analogen Ausgang des	10
6	Digitalauslesen	Erfassung der Atemfrequenz über den digitalen Ausgang des	8
7	Helmmontage	3D-Druck und Montage aller Sensoren am Helm	5
8	Schaltungsaufbau	Aufbau auf dem Steckbrett, Lösung von Stromversorgungspro	6
9	LED-Softwaretest	Gemeinsames Testen der LED-Steuerungssoftware mit Napol	7
10	Recherche	Analyse von Code-Beispielen und Einarbeitung in die Program	18
11	Dokumentation	Erstellung von Projektbericht, Projektbeschreibung und Präse	21
12	Abschluss tests	Durchführung abschließender Tests; Anpassung von Schaltun	6
13	Integration	Ausführung Lötarbeiten und Verkabelung	13
14	Front-LED	Anpassung der LED-Software an die Front-LEDs-Streife	4
15	Mikrophon Aufbau	lösung Aufbau des Mikrofons suchen/testen	8
16	Ergenisvorstellung	Vorstellung und Ausführung	3
17	pair Programming	Ein- und Ausatmen mit Vinzenz Bauman	5
18	Entwicklungsumgebung	Neues Projekt Setzen, Umstieg auf Vscod, Bibliotheken hoch	2
19			
20			
21			
22			
23			
24			
25			
26			
27			
28			
29			
30			
31			
32			
33			
34			
35			
36			
37			
38			
39			
40			
41			
42			
43			
Gesamt:			152

Abbildung 20: Stundenliste Ronny Kenfack

7.4 Stundenliste Nzeko Prince

Stundenliste

Name: Prince Napoleon Nzeko Wagueu
 Gruppe: 9 VitaHelm

Pos.	Bezeichnung	Beschreibung	Stunden in h
1	Komponentenrecherche und	Recherche und Suche passender Bauteile online und im Labor	4
2	Auswahl & Dokumentation de	Auswahl der LED-Komponenten, Erstellung der Bauteilliste	3
3	Projektpräsentation & Vorstel	Projektbeschreibung mit Vorstellung vor den Professoren und Kommilitonen	8
4	Dokumentationsstruktur & Git	Dokumentation, Vorbereitung der Vorlagen für das Projekt auf Git	6
5	Aufbau & Test der LED-Hardw	Testaufbau der LED-Streifen mit ESP32, Prüfung und Bereitstellung der Stro	5
6	Bibliotheksintegration & erste	Integration von Bibliotheken und erste Farbausgabe durch Programmcode	4
7	Elektronikintegration in den H	3D-Druck, LED-Lötarbeiten	8
8	Elektronikintegration in den H	Mitarbeit bei der Integration der Sensoren und Bauteile in den Helm mit Ve	10
9	Debugging der Atemerkennun	Mitarbeit bei der Problemlösung des Ein- und Ausatmens	4
10	Koordination & Teambesprecht	Teambesprechungen	8
11	Inbetriebnahme der LED-Streif	Aufbau und Ansteuerung beider LED-Streifen (Haupt + Front)	6
12	Farbinterpolation & Vitaldata	Entwicklung der Farbübergangslogik, Einbau der Atmung- und Herzschlag-S	8
13	Definition der Farblogik & Mo	Abstimmung mit Team zur moodValue-Definition, Mapping und Farblogik	2
14	Codeorganisation & Versionsk	Code-Strukturierung und Aufteilung (.h/.cpp), Integration ins gemeinsame F	3
15	Analyse der Sensorschnittstell	Sensorverständnis (Mikrofon, Oxymeter)	11
16	Integrationstest mit Live-Date	Gemeinsame Tests mit echten Sensorwerten (Atmungs- und Herzfrequenz)	25
17	Optimierung der LED-Logik	Anpassung + Korrektur der LED-Funktionen	13
18	Abschlusspräsentation & Dem	Projektpräsentation, Ergebnisvorführung mit Vorbereitung	13
19	Abschlussdokumentation & B	Dokumentation	12
20			
21			
22			
23			
24			
25			
26			
27			
28			
29			
30			
31			
32			
33			
34			
35			
36			
37			
38			
39			
40			
41			
42			
43			
Gesamt:			153

Abbildung 21: Stundenliste Napoleon Nzeko

Abbildungsverzeichnis

1	Datenflussdiagramm	1
2	Aufgabenverteilung auf die einzelnen Komponenten	2
3	Verwendete Protokolle	4
4	VitaHelm	6
5	Power Supply und Schaltungsbild	7
6	Anschlüsse der LEDs und Sensoren an esp	8
7	LEDs und Photosensor Oxymeter	9
8	MiKrofon KY-038	10
9	Beschleunigungssensor ADXL345	10
10	Ai Thinker ESP32 Cam Module links	11
11	OV2640 Kamera mit 160 Grad Weitwinkel	11
12	WS2812B-LEDs	12
13	Ablauf der Erregungswertberechnung	19
14	Summe des Erregungswertes mit der versteckten Gewichtungen	24
15	GUI	25
16	Power Supply und Schaltungsbild	26
17	Mikrofon Aufbau	27
18	Stundenliste Vinzenz Baumann	31
19	Stundenliste Albero König	32
20	Stundenliste Ronny Kenfack	33
21	Stundenliste Napoleon Nzeko	34

Tabellenverzeichnis

1	Verwendete Bauteile im Projekt	3
2	Erregungslevel und zugehörige LED-Farben	17

Literatur

- Dombrowski, Peter (1999). „Wege in euklidischen Ebenen“. In: *Wege in euklidischen Ebenen Kinematik der Speziellen Relativitätstheorie: Eine Auswahl geometrischer Themen mit Beiträgen zu deren Ideen-Geschichte*. Berlin, Heidelberg: Springer Berlin Heidelberg, S. 1–158. ISBN: 978-3-642-58501-2. DOI: 10.1007/978-3-642-58501-2_1. URL: https://doi.org/10.1007/978-3-642-58501-2_1.
- Heikenfeld, Jason u. a. (2018). „Wearable sensors: modalities, challenges, and prospects“. In: *Lab on a Chip* 18.2, S. 217–248. DOI: 10.1039/C7LC00914C. URL: <https://doi.org/10.1039/C7LC00914C>.
- Lee, Sangkyun u. a. (2024). „Dynamic link between respiration and pupil dynamics during arousal transitions“. In: *Nature Communications* 15, S. 4567. DOI: 10.1038/s41467-024-4567-1. URL: <https://www.nature.com/articles/s41467-024-04567-1>.

- SparkFun Electronics (2019). *SparkFun MAX3010x Pulse and Proximity Sensor Library*. https://github.com/sparkfun/SparkFun_MAX3010x_Sensor_Library. Version 1.2.9. Accessed: 2025-07-14.
- StatPearls (2022). „Physiology, Respiratory Rate“. In: *StatPearls [Internet]*. Typical respiratory rate in resting adults: 12–20 breaths per minute. URL: <https://www.ncbi.nlm.nih.gov/books/NBK537306/>.
- Tanaka, H, KD Monahan und DR Seals (Jan. 2001). „Age-predicted maximal heart rate revisited“. In: *Journal of the American College of Cardiology* 37.1, S. 153–156. ISSN: 0735-1097. DOI: 10.1016/s0735-1097(00)01054-8. URL: [https://doi.org/10.1016/s0735-1097\(00\)01054-8](https://doi.org/10.1016/s0735-1097(00)01054-8).
- Wickens, Christopher D. u. a. (2019). *An Introduction to Human Factors Engineering*. 3. Aufl. Boston: Pearson. ISBN: 9780134729329.

8 Anhang

8.1 Quellen

https://cdn-reichelt.de/resize/600%2F-/web/xxl_ws/A300%2FDEBO_3AXIS_GY291_02.png?type=ProductXxl&resize=600%252F-&